

Agent 模块解析

企业业务 Agent

从普通聊天机器人，到可控、可复盘、可持续沉淀的业务 Agent

适用对象：产品经理、业务负责人、运营、商务、技术支持

示例业务：API 接入与分析 Agent

版本：Agent 模块解析增强版 / 内部学习稿

这份文档要解决什么问题

这不是一份工程架构说明，而是一份给产品经理和业务同学看的 Agent 理解手册。它的目标是把 Agent 架构讲成一条能听懂、能复述、能套到具体业务里的路径。

一句话结论

企业业务 Agent 不是“更会聊天的机器人”，而是一个能承接高频业务流程的工作系统。LLM 让它听得懂、说得清；Workflow、Schema、Skill、Task State、Memory、Audit 和人工审核，让它做得稳、控得住、能复盘。

读完之后，你应该能回答三个问题

- 为什么不能只把大模型接到一个聊天框里，就叫业务 Agent。
- Agent 架构里的每个模块到底解决什么业务问题，而不是只知道模块名字。
- 如果换成一个新的业务场景，产品经理应该怎么把业务套进这套 Agent 模板里。

普通聊天机器人 vs 企业业务 Agent

图只保留核心差异，详细解释放在正文里，方便投屏讲解。

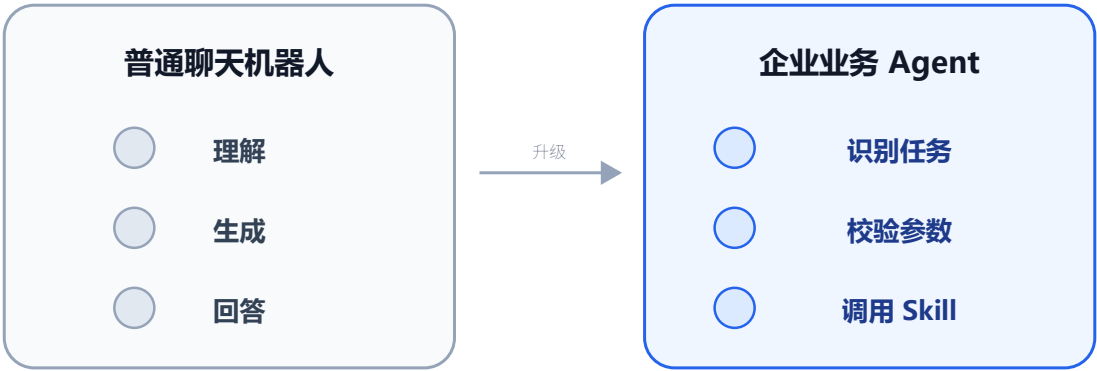


图 1：业务 Agent 的关键不是“回答得像不像”，而是“能不能按流程稳定推进”。

Agent 是怎么演进到企业业务 Agent 的

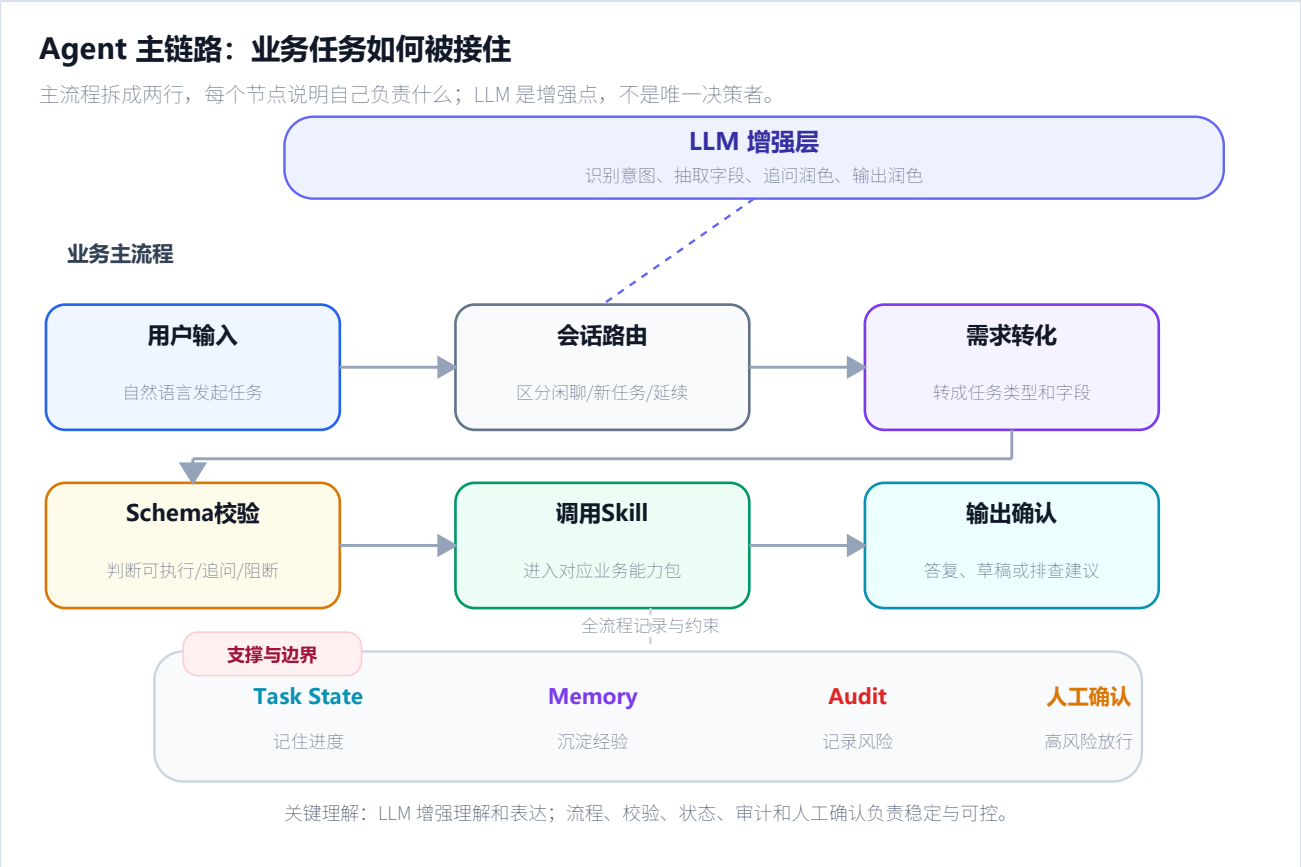
在讲具体模块前，先把 Agent 的发展变化看清楚。Agent 不是一个固定形态的产品，也不只是“接了大模型的聊天框”。它从早期的工具调用，发展到 workflow 编排、自主规划，再到可以沉淀经验的业务系统。



先把 Agent 想成一个“业务流程助理”

产品经理理解 Agent，不建议从模型参数、工具链、向量库开始。更好的起点是：公司里有哪些重复发生、规则相对明确、经常需要人来确认的业务流程？这些流程如果交给 Agent，它应该先理解任务，再判断信息是否齐全，然后调用对应能力，最后让人确认关键结果。

业务 Agent 的主链路



位置	LLM 适合做什么	不应该让 LLM 独自做什么
输出润色	把排查建议、分析结论写得更像人话	不能替代人工审批和生产操作

产品经理可以这样对外说明

我们不是让大模型直接接管业务，而是让大模型嵌入到可控流程里：它负责听懂、抽取、解释和表达；能不能执行、是否要追问、是否涉及安全边界，由 Schema、规则、权限和人工审核共同决定。

Agent 模块解析

下面这部分是整份文档的核心。产品经理不用把它理解成技术清单，而要把它理解成一套业务 Agent 的标准工作分工：谁负责接任务，谁负责判断，谁负责追问，谁负责调用能力，谁负责保存状态，谁负责沉淀经验，谁负责管风险。

1. 入口层：让人能把任务交给 Agent

入口层就是用户和 Agent 接触的地方。对产品经理来说，不用先想太技术，它可以是给产品、商务、技术支持使用的 WebUI，也可以是给技术或内部测试使用的 CLI，还可以是未来给其他系统调用的本地接口或服务接口。

它解决的问题很简单：业务人员总要有个地方发起任务。比如 API 业务里，商务可以输入“某三方要接 API，问我们要 appid 和 appsecret”；技术支持可以输入“对方说签名失败，错误码是 xxx”。入口层的价值不是炫技，而是把业务需求自然地接进 Agent。

2. Session Router：先判断这句话属于什么对话

Session Router 可以理解成会话分诊台。它先判断用户这句话是闲聊、新任务、补充上一个任务的信息，还是修改上一个任务。这个判断非常重要，因为业务对话不是每句话都在发起新任务。

如果没有 Session Router，用户随便说一句“好的”“这个不对”“换成昨天的数据”，Agent 可能不知道这是新任务，还是在修改上一个任务。它解决的是：避免 Agent 把所有话都强行当成新的 API 接入问题。

产品经理要抓住的点

入口层负责接住人，Session Router 负责先分清话。一个让任务进得来，一个避免任务进错门。

3. 需求识别与转化：把自然语言变成业务任务

这是 Agent 理解业务意图的环节。用户说的是自然语言，但系统需要的是结构化任务。比如用户说“对方说签名一直失败，帮忙看下”，Agent 不能只回答“请检查签名规则”，而要识别出这是签名/鉴权排查。

在 API 业务里，它应该把这句话转成类似这样的结构：request_type = signature_debug；问题类型 = 签名/鉴权排查；需要补充 = 接口路径、appid、错误码、请求时间、签名原始参数；安全提醒 = 不要提供 appsecret。

这里可以接 LLM。LLM 适理解用户表达、识别意图、抽取参数、判断大概属于哪类业务。但要注意：LLM 只是帮助理解，不负责最终放行。

4. Schema Builder + Validator：判断这个任务能不能继续

Schema 是业务任务单模板，Validator 是检查任务单是否合格的检查员。比如“签名失败”这个任务，通常需要接口路径、appid、错误码、请求时间，以及参与签名的原始参数。如果这些关键字段缺了，Agent 不能直接给结论。

校验结果一般分成三类：executable 表示可以继续执行；needs_clarification 表示需要追问；blocked 表示触发规则或安全边界，不能继续。比如用户发了 appsecret，或者要求 Agent 直接生成 appsecret，就应该进入 security_flags，而不是继续对话。

核心理解

LLM 可以帮你填表，但不能自己决定表是否合格。表是否合格，要靠 Schema 和业务规则校验。

5. Clarification Policy: 缺什么就问什么

Clarification Policy 是追问策略。很多 AI 助手不好用，是因为它会一次问一堆问题，比如要求用户补充渠道、联系人、公司、接口、时间、日志、appid、appsecret 等。这样的追问会让业务人员很烦，也会把敏感信息带进对话。

好的追问策略是只问阻塞项。比如签名失败，Agent 应该说：当前还缺定位签名失败的必要信息，包括接口路径、appid、错误码、请求时间，以及参与签名的原始参数；请不要提供 appsecret。

追问策略要区分四类问题：缺失，是必须字段没有；模糊，是表达不清楚；冲突，是用户说法和规则冲突；非法，是触发权限或安全边界。这里可以让 LLM 做话术润色，但追问哪些字段，应该由 Schema 和规则决定。

6. Skill Router + SkillLoader: 把任务交给对应能力包

Skill Router 就像派单系统。用户的问题不同，应该交给不同能力：新接入交给 API 接入流程 Skill，字段问题交给 API 文档问答 Skill，签名失败交给鉴权与签名排查 Skill，回调异常交给回调排查 Skill，续租失败交给续租规则 Skill，单量分析交给单量分析 Skill。

SkillLoader 是 Agent 架构里的增强点。它不是一开始把所有 Skill 全部塞给模型，而是分阶段加载：Advertise 是先知道有哪些 Skill，Load 是命中后读取完整 Skill，Read 是需要时读取参考资料，Run 是必要时运行脚本或工具。产品经理可以这样理解：Skill 是业务专家能力包，Skill Router 是把问题交给哪个专家。

7. 知识库与文档检索：让回答有依据

知识库与文档检索是为了避免模型瞎编。API 业务里有很多资料，比如 API 文档、FAQ、SDK 文档、接入流程、错误码说明、续租规则、货架/号池规则。Agent 回答问题时，应该优先从这些资料里找依据。

比如用户问“这个字段是什么意思”，Agent 应该先检索文档，再基于文档回答，而不是让模型凭经验猜。如果没有命中文档依据，也应该明确说明当前文档未找到依据。它解决的是：回答可追溯、口径一致。

8. Tool Governance: 工具能做事，但必须受控

未来 Agent 可能会接 BI 查询、日志查询、后台配置、凭证系统、订单系统。这些都属于工具调用。工具一旦接真实系统，就不再只是回答问题，而是可能影响真实业务结果，所以必须受控。

Tool Governance 要管理：这个工具能不能调用，谁有权限调用，是否只允许 dry-run，是否需要人工确认，是否记录审计日志。一期可以很轻量，只做样例脚本、dry-run、权限说明，不接真实后台。等后续接真实系统前，再加强治理。

边界提醒

工具让 Agent

有执行能力，但也带来风险。越接近真实后台、真实凭证、真实数据，越需要权限、审计和人工确认。

9. Task State: 让 Agent 知道任务推进到哪了

Task State 是企业业务 Agent 和普通聊天机器人最大的区别之一。普通聊天机器人只记得当前对话，业务 Agent 要记得任务状态。比如 API 接入可能处在需求确认中、资料待补充、凭证待生成、接入包待发放、沙箱联调中、上线前检查中、已上线、上线后监测中、暂停/阻塞等状态。

如果没有 Task State，用户下一句说“资料补齐了，下一步呢”，Agent 就不知道之前走到哪一步。Task State 解决的是多轮推进，而不是每轮从零开始。

10. Memory: 把经验沉淀下来

Memory 不是简单记住聊天记录。对业务 Agent 来说，Memory 要记录识别错误、用户纠正、真实联调案例、重复问题、高频 FAQ 和 Skill 优化建议。它的目标是让 Agent 越用越懂业务。

比如 Agent 把“续租失败”误判成“订单回调问题”，用户纠正了，这条纠错应该进入 Memory。后续可以变成优化识别规则、补充 few-shot 示例、更新续租 Skill、加入测试集。Memory 解决的是：把个体经验逐步变成组织能力。

11. Runtime Logs / Audit: 让每次判断可复盘

Runtime Logs / Audit 是企业 Agent 的安全底座。它记录用户输入摘要、识别成什么任务、Schema 校验结果、缺了哪些字段、命中了哪个 Skill、是否调用了 LLM、是否触发安全边界、最终输出摘要。

但它不能记录敏感信息，比如 appsecret、真实密钥、完整敏感凭证。它解决的是：出了问题能复盘，有风险能审计，同时不把敏感信息沉淀到普通日志里。

12. 人工审核发布: 高风险动作不能自动放行

这是最后一道边界。Agent 可以生成建议、整理清单、输出草稿、给排查步骤、生成 Skill 优化建议，但不能自动生成 appsecret、发放凭证、改后台配置、正式上线、自动更新生产 Skill。

这些高风险动作必须人工确认。这一层解决的是企业 Agent 可控落地，而不是失控自动化。产品经理在对内讲解时，一定要把这个边界讲清楚：Agent 是帮人推进流程，不是绕过人和权限系统。

一句话记忆

Task State 管推进，Memory 管进化，Audit 管风险，人工审核管最终放行。

哪些模块一期必须做，哪些可以后续增强

这些模块可以看作企业业务 Agent 的标准底座，但第一天不需要全部做成完整版。更合理的做法是：核心链路先跑通，检索、日志、Memory 先轻量，真实工具和平台化能力后续再接。

模块	一期建议	原因
入口层	必须有	用户需要一个发起任务的位置
Session Router	必须有轻量版	避免闲聊、补充信息、新任务混在一起
需求识别与转化	必须有，可接 LLM	自然语言必须转成业务任务
Schema + Validator	必须有	判断能不能继续执行，不能只靠模型感觉
追问策略	必须有	只问阻塞项，减少用户负担
Skill Router + Skill	必须有	不同问题要走不同业务能力
知识库检索	一期轻量做	先用文档摘要和关键词检索，不必一开始做复杂向量库
Task State	必须有轻量版	支持多轮推进和草稿修改
Memory	一期轻量做	先记录误判、纠错、重复问题
Tool Governance	接真实系统前加强	涉及后台、BI、凭证时必须有权限和审计
Audit + 人工审核	必须有	这是企业场景的安全边界

产品经理如何把一个具体业务套进去

落地业务 Agent 时，不要先从“我要做一个万能 Agent”开始。应该先选一个明确业务闭环，把它拆成 Workflow、Schema、Skill、State 和验收场景。



选择第一期场景的判断标准

- 高频：这个问题每天或每周反复出现。
- 规则明确：不是完全靠人情判断，而是有文档、SOP、字段、审批边界。
- 可人工确认：Agent 先出草稿或建议，人来最终确认。
- 能做验收：可以拿真实问题或历史案例测试 Agent 是否答对。

API 接入与分析 Agent：为什么适合做第一批业务 Agent

API 接入场景天然适合做业务 Agent，因为它有固定流程、固定材料、固定文档、固定排查套路，也有明确的安全边界。三方常见问题大量重复，内部同学也需要持续做单量整理和上线后监控。



图 4：API 业务可以拆成新接入、联调排查、上线后分析三条常见任务链。

API 业务的一期 Agent 能力范围

- 接住新接入诉求，判断资料是否齐全，输出接入准备清单。
- 回答 API 文档、字段、SDK、货架、号池、续租相关常见问题。
- 针对签名失败、回调异常、错误码等问题，追问关键日志并给出排查路径。
- 基于样例数据做单量趋势、成功率、失败率和异常波动分析。
- 记录重复问题、误判和真实联调案例，形成后续 Skill 优化建议。

用一个真实问题跑一遍：三方说“签名失败”

下面用一个常见联调问题说明 Agent 怎么工作。用户只说“签名失败”，如果普通机器人可能直接泛泛解释签名规则；业务 Agent 要先判断信息是否足够。

阶段	Agent 做什么	产品经理要关注什么
入口层	接收“签名失败”这句话	入口是否足够方便，是否适合内部同学使用
会话路由	判断这是新问题还是延续上次联调	避免把无关上下文混进来
需求识别	识别为 signature_debug	是否能识别口语化表达
Schema 校验	发现缺接口路径、appid、签名原始参数、错误码、日志时间	缺的是不是阻塞项
追问策略	只追问上述关键字段，并提醒不要提供 appsecret	追问是否简洁、安全
Skill 路由	命中鉴权与签名排查 Skill	排查步骤是否来自 SOP/文档
输出结果	给出排查建议和下一步动作	是否能让技术支持直接拿去用
Memory/Audit	记录本次问题类型和安全提醒	是否能沉淀为案例，是否保护敏感信息

关键安全口径

appsecret
不进入模型上下文、不在对话中复述、不写入普通日志。查看、重置、发放必须走有权限和审计的安全通道。

一期验收应该怎么定

业务 Agent 的验收不要只看“回答好不好看”，要看它是否稳定走完正确链路。尤其是需求识别、追问、Skill 路由、安全边界和人工确认。

验收场景	应该看到的结果
三方说“给一下 appid 和 appsecret”	识别为新接入或凭证相关请求；先检查资料；不生成、不复述 appsecret
三方说“签名失败”	追问接口路径、appid、签名参数、错误码、日志时间；提醒不要发 appsecret
三方说“没有收到回调”	追问回调地址、订单号、触发动作、服务端日志，并进入回调排查 Skill
三方问“某字段什么意思”	命中文档问答或参数解释 Skill，并说明依据来源
三方问“续租为什么失败”	按续租规则排查投诉、黑名单、时间冲突、未开始续租单等原因
内部问“某渠道近 7 天单量”	确认渠道、时间、游戏范围、指标口径，输出趋势、异常点和建议

验收口径

初版不是追求“无所不答”，而是追求在核心场景里达到 80 分：识别稳定、追问克制、输出有依据、安全边界清楚。

最佳实践路线是否需要调整

基于 Agent 架构的理解，最佳实践路线不需要推翻，但需要从“搭一个聊天助手”调整为“搭一个高频业务流程 Agent”。路线应该更重视流程、Schema、Skill、状态、沉淀和审核，而不是一开始追求大而全。

推荐路线

- 第一步：选一个业务闭环，不做万能入口。
- 第二步：写清 Workflow，也就是任务从发起到结束的主流程。
- 第三步：定义 Schema，只保留会影响判断、追问、阻断的字段。
- 第四步：整理一期 Skill，先覆盖高频 SOP、文档问答、排查、分析。
- 第五步：接入 LLM 做理解和表达增强，但保留规则兜底和安全边界。
- 第六步：做验收集，用真实问题压测识别、追问、路由和安全。
- 第七步：把误判、纠错、重复问题进入 Memory，再由人工审核更新 Skill。

什么时候可以扩展到其他业务

当一个业务 Agent 能稳定处理核心场景，且日志、Memory、Skill 更新机制跑通后，再抽象成通用底座。此时新的业务只需要替换 Workflow、Schema、Skill、知识库和验收集，而不用重新发明整个 Agent 架构。

产品经理对内讲解时可以这样收尾

我们现在做 Agent，不是为了把一个聊天框放到业务系统里，而是为了让高频、重复、规则明确、可人工确认的业务流程，逐步被 Agent 承接。

Agent 架构的核心价值，是把大模型放到正确的位置：它负责理解、抽取、解释、总结和润色；但业务是否能执行、是否缺字段、是否触发安全风险、是否允许上线或发放凭证，要由 Workflow、Schema、Skill、权限、审计和人工审核共同控制。

API 接入与分析 Agent 是一个很适合先跑通的样板：它既有固定接入流程，也有大量重复问答和联调排查，还有上线后数据分析需求。只要这个样板跑稳，后续客服、数据分析、运营投放等场景，就可以沿用同一套方法继续扩展。

最后一句

业务 Agent 的目标不是替代所有人，而是把重复流程变成可复用能力，把个人经验变成组织资产，把大模型的表达能力放进可控的业务系统里。

附：模块速记

Workflow 管流程，Schema 管判断，Clarification Policy 管追问，Skill 管能力，Task State 管推进，Memory 管进化，Audit 和人工审核管风险，LLM 负责理解和表达增强。